

实验九 指针程序设计（一）指针与数组

一、 实验目的：

1. 理解变量、地址与指针之间的关系，掌握取地址运算与指向运算；
2. 理解数组名与数组元素地址之间的关系，掌握数组名作为函数参数的程序编写；
3. 了解约瑟夫环这一经典数学问题应用，尝试通过数组或者指针编程实现约瑟夫环求解。

二、 实验要求：

1. 用 C 语言实现源程序的编写；
2. 源程序应书写规范，适当添加注释；
3. 实验课前完成实验内容源程序的初稿；实验课对源程序进行编辑、调试、运行；实验课后尽快完成实验报告，并按规定时间上交实验报告。

三、 实验内容：

1. 在主程序通过键盘输入两个正整数，编写并调用自定义函数 void swap(int *x, int *y) 实现两个整数变量值的交换并输出交换后的结果。

```
##include <stdio.h>

void swap(int *n, int *m){ //注意要传入整形变量的指针
    int tmp = *n;
    *n = *m;
    *m = tmp;
}

int main(int argc, char *argv[]) {
    int n, m;
    scanf("%d %d", &n, &m);

    swap(&n, &m);

    printf("交换之后: %d %d", n, m);
}
```

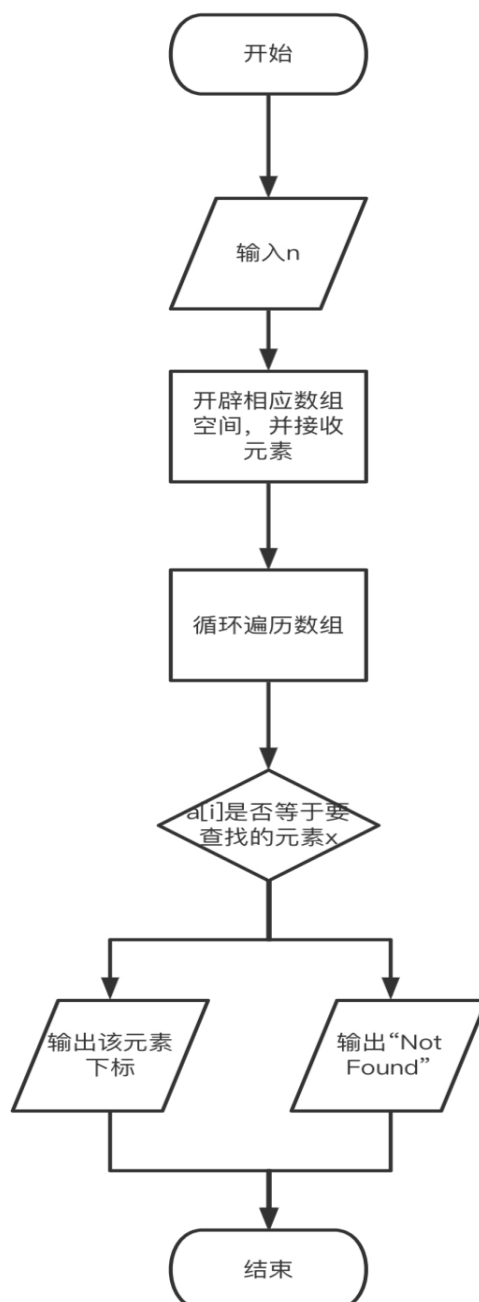
测试结果：

1 2

交换之后：2 1

2. 输入一个正整数 n ($1 < n \leq 10$), 然后输入 n 个整数存入数组 a 中, 再输入一个整数 x , 在数组 a 中查找 x , 如果找到则输出相应元素的最小下标, 否则输出 "Notfound". 要求定义并调用函数 `int search(int *list, int n, int x)`, 它的功能是在数组 `list` 中查找元素 x , 若找到则返回相应元素的最小下标, 否则返回 -1 (本题要求在实验报告中画出流程图)。

程序流程图:



```

#include <stdio.h>
#include <stdlib.h>

int search(int *list, int n, int x){
    for(int i = 0; i < n; i++){
        if(list[i] == x){
            printf("元素%d 的最小下标:%d", x, i);
            return i;
        }
    }
    printf("Notfound!");
    return -1;
}

/*
6
8 21 74 12 64 81
*/
int main(int argc, char *argv[]) {
    int n;
    scanf("%d", &n);
    //开辟相应的一维数组空间 (地址 -> 空间的转换)
    int *a = (int *) malloc(n * sizeof(int ));
    for(int i = 0; i < n; i++)
        scanf("%d", &a[i]);
    search(a, n, 64);
}

```

测试结果:

```

6
8 21 74 12 64 81
元素64的最小下标:4

```

3. 输入两个正整数 n 和 m ($1 < m < n \leq 50$), 有 n 个人围成一圈, 按顺序从 1 到 n 编号。从第一个人开始报数, 报数 m 的人退出圈子, 下一个人从 1 开始重新报数, 报数 m 的人退出圈子。如此循环, 直到留下最后一个人。请按退出顺序输出退出圈子的人的编号, 以及最后一个人的编号。提示: 将每个人的编号存入数组, 从第一个人开始报数, 输出报数 m 的人的编号, 并将该编号清除为 0, 重复这样的操作直至只剩下一个不为 0 的数, 该数就是最后一个人的编号。

```

#include <stdio.h>
#include <stdlib.h>

int order[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

int getIndex(int *a, int n, int x){
    for(int i = 0; i < n; i++){
        if(a[i] == 0)
            break;
        if(a[i] == x)
            return i; //返回索引
    }
    return -1;
}

void solve(int *a, int n, int m){ //间隔为m的人退出数组
    int interval = m - 1;
    int cnt = -1;
    for(int i = 0; i < n; i++){ //每次循环出队一个人
        int pos;

        if(i == n - 1) //出口
            break;
        // while(!inArray(a, n, a[i]))
        int index;
        pos = i;
        if(a[i] == 0){
            index = i;
            pos = getIndex(a, n, order[index]);
            // if(pos == -1) //未找到, 找下一个元素
        }
        pos = (pos + interval) % (n - i); //模运算实现循环(循环队列的思想)
        printf("%d 号出队\n", a[pos]);
        //剩下元素 向前移动即可
        for(int j = pos; j < n - i; j++) //插入排序的思想, 元素后移~ (注意动态变化)
            a[j] = a[j + 1];
        // for(int i = 0; i < n; i++)
        //     printf("%d ", a[i]);
        // printf("\n");
    }
    printf("最后胜出的玩家的号码:%d", a[0]);
}

```

```

}
//6 3
int main(int argc, char *argv[]) {
    int n, m;
    while(1){ //注意符合范围哈~
        scanf("%d %d", &n, &m);
        if(n >= 1 && m <= 50 && n >= m) {
            printf("录入成功! \n");
            break;
        }else {
            printf("录入失败, 请重新输入! \n");
        }
    }
    int *a = (int*) malloc(n*sizeof(int));
    for(int i = 0; i < n; i++) //记得给 a 数组初始化~
        a[i] = order[i];
    solve(a, n, m);
}

```

测试结果:

```

6 3
录入成功!
3号出队
5号出队
1号出队
2号出队
6号出队
最后胜出的玩家的号码:4

```

四、 实验小结:

1. 通过本次试验的怜惜, 深刻理解了变量、地址与指针之间的关系, 掌握了取地址运算与指向运算, 取地址运算即获取某变量的内存地址, 指向运算即通过地址变量, 简介访问想要访问的变量, 即指针;

2. 通过 malloc 函数动态分配内存，深刻理解了数组名与数组元素地址之间的关系，掌握了数组名作为函数参数的程序编写，同时明确了对于基地址、首元素等概念的联系。

3. 了解了约瑟夫环这一经典数学问题应用，最终实现了利用数组和指针，进行编程求解约瑟夫环。

五、 实验成绩：